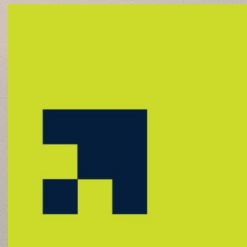


Functions

Dr. Sanjit Kumar Saha



Gisma
University
of Applied
Sciences





Functions

- A *function* is a block of code which only runs when it is called
- A *function* can return data as a result
- A *function* helps avoiding code repetition
- *Functions* are one way to break an application down into more manageable units

$$f(x) = x^2$$

$$f(3) = 3^2$$

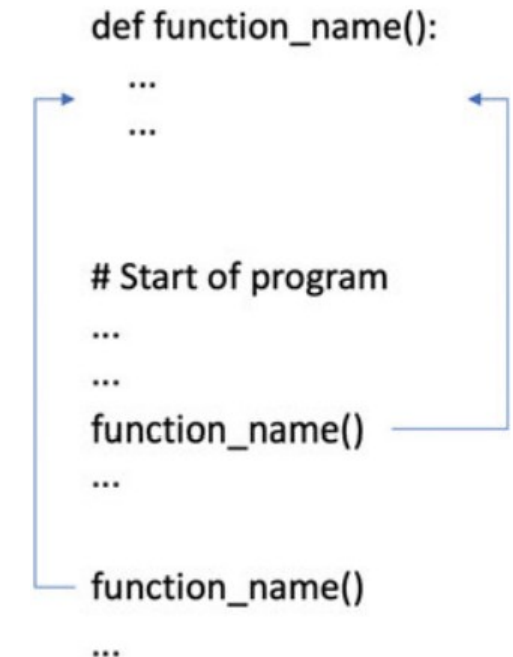


How Functions Work

[Hunt 2019]

3

- When a function is called (or invoked)
 - The flow of control a program jumps from where the function was called to the point where the function was defined
 - All the values that were in place when the function was called are stored away
 - On something called the stack
 - If the function defines its own versions of the variables, they do not overwrite each other
 - The body of the function is then executed before control returns back to where it was called from





Types of Functions

- Built-in functions
 - Are those provided by the language
 - We have seen several of these already
 - e.g., *print()* and *abs()*
- User-defined functions
 - Are those written by developers
 - We have to either import or define them



Importing Functions from Modules

- Most functions that are built into the Python language are stored in a collection of functions called a module
 - Consider a module to be the same as a code library
- An import statement is used to provide access to a module
 - Importing the whole module
 - `import math`
 - `math.exp(2)`
 - Importing only parts of a module
 - `from math import exp`
 - `exp(2)`

$$f(x) = e^x$$

$$f(2) = e^2$$



Defining Functions

- A function is defined using the `def` keyword, followed by a function name and parentheses:

```
def my_function():  
    print("Hello from a function")
```

- This creates a function named `my_function` that prints "Hello from a function" when called



Defining Functions

- To call a function, write its name followed by parentheses:

```
def my_function():  
    print("Hello from a function")
```

```
my_function()
```

- Function names follow the same rules as variable names in Python:
 - A function name must *start with a letter* or *underscore*
 - A function name can *only contain letters, numbers, and underscores*
 - Function names are *case-sensitive* (*myFunction* and *myfunction* are different)



Why Use Functions?

Repetitive code:

```
temp1 = 77
celsius1 = (temp1 - 32) * 5 / 9
print(celsius1)

temp2 = 95
celsius2 = (temp2 - 32) * 5 / 9
print(celsius2)

temp3 = 50
celsius3 = (temp3 - 32) * 5 / 9
print(celsius3)
```

Reusable code:

```
def fahrenheit_to_celsius(fahrenheit):
    return (fahrenheit - 32) * 5 / 9

print(fahrenheit_to_celsius(77))
print(fahrenheit_to_celsius(95))
print(fahrenheit_to_celsius(50))
```

With functions, you write the code once and reuse it



Return Values

- Functions can send data back to the code that called them using the *return* statement.
- When a function reaches a *return* statement, it stops executing and sends the result back:

```
def get_greeting():  
    return "Hello from a function"
```

```
message = get_greeting()  
print(message)
```



Function Arguments

- Information can be passed into functions as arguments.
- Arguments are specified after the function name, inside the parentheses.
 - We can add as many arguments as you want, just separate them with a comma.

```
def my_function(fname):  
    print(fname + " Refsnes")
```

```
my_function("Emil")  
my_function("Tobias")  
my_function("Linus")
```



Parameters vs Arguments

- A *parameter* is the variable listed inside the parentheses in the function definition.
- An *argument* is the actual value that is sent to the function when it is called.

```
def my_function(name): # name is a parameter  
    print("Hello", name)
```

```
my_function("Emil") # "Emil" is an argument
```



Parameters vs Arguments

[Hunt 2019]

12

```
def my_function(fname, lname):  
    print(fname + " " + lname)
```

```
my_function("Emil")
```

Output?



Default Parameter Values

- We can assign default values to parameters.
- If the function is called without an argument, it uses the default value:

```
def my_function(country = "Norway"):  
    print("I am from", country)
```

```
my_function("Sweden")  
my_function("India")  
my_function()  
my_function("Brazil")
```




Keyword Arguments

- We can send arguments with the *key = value* syntax

```
def my_function(animal, name):  
    print("I have a", animal)  
    print("My", animal + "'s name is", name)
```

```
my_function("dog", "Buddy")
```

```
def my_function(animal, name):  
    print("I have a", animal)  
    print("My", animal + "'s name is", name)
```

```
my_function(animal = "dog", name = "Buddy")
```



Passing Different Data Types

- We can send any data type as an argument to a function (string, number, list, dictionary, etc.).
- The data type will be preserved inside the function:

```
def my_function(fruits):  
    for fruit in fruits:  
        print(fruit)
```

```
my_fruits = ["apple", "banana", "cherry"]  
my_function(my_fruits)
```



Example: Factorial Function

[Hunt 2019]

16

```
def factorial(n):  
    result = 1  
    for i in range(1, n + 1):  
        result *= i  
    return result
```



Arbitrary Arguments

[Hunt 2019]

17

- If we do not know how many arguments will be passed into your function, add a ***** before the parameter name.
- This way, the function will receive a tuple of arguments and can access the items accordingly:

```
def greeter(*args):  
    for name in args:  
        print("Welcome", name)
```

```
greeter("John")  
greeter("John", "Alice")
```

- The ***args** parameter allows a function to accept any number of positional arguments.
- Inside the function, args becomes a tuple containing all the passed arguments



Arbitrary Arguments

[Hunt 2019]

18

```
def my_function(greeting, *names):  
    for name in names:  
        print(greeting, name)
```

```
my_function("Hello", "Emil", "Tobias", "Linus")
```

Output?



Keyword Arguments

- If we do not know how many keyword arguments will be passed into your function, add two asterisks ****** before the parameter name.
- This way, the function will receive a dictionary of arguments and can access the items accordingly:

```
def my_function(**kid):  
    print("His last name is " + kid["lname"])  
  
my_function(fname = "Tobias", lname = "Refsnes")
```



Keyword Arguments

```
def my_function(**myvar):  
    print("Type:", type(myvar))  
    print("Name:", myvar["name"])  
    print("Age:", myvar["age"])  
    print("All data:", myvar)
```

```
my_function(name = "Tobias", age = 30, city = "Bergen")
```




Combining `*args` and `**kwargs`

- We can use both `*args` and `**kwargs` in the same function.
- The order must be:
 - regular parameters
 - `*args`
 - `**kwargs`

```
def my_function(title, *args, **kwargs):  
    print("Title:", title)  
    print("Positional arguments:", args)  
    print("Keyword arguments:", kwargs)
```

```
my_function("User Info", "Emil", "Tobias", age = 25, city = "Oslo")
```



Function Inside Function

```
def myfunc():  
    x = 300  
    def myinnerfunc():  
        print(x)  
    myinnerfunc()  
  
myfunc()
```



Anonymous Functions

[Hunt 2019]

<https://towardsdatascience.com/how-to-effectively-use-lambda-functions-in-python-as-a-data-scientist-fd6171554053>

- Sometimes, we want to create a function and use it only once
- An anonymous function does not have a name and can only be used at the point that it is defined
 - A *lambda function* is a small anonymous function
 - A *lambda function* can take any number of arguments, but can only have one expression

lambda arguments: expression

Add 10 to argument a, and return the result:

```
x = lambda a : a + 10  
print(x(5))
```



Lambda Functions

[Hunt 2019]

<https://towardsdatascience.com/how-to-effectively-use-lambda-functions-in-python-as-a-data-scientist-fd6171554053>

24

```
x = lambda a, b : a * b  
print(x(5, 6))
```

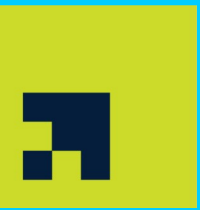
Output?



Recursion

- Recursion is when a function calls itself
- Every recursive function must have two parts:
 - A *base case* - A condition that stops the recursion
 - A *recursive case* - The function calling itself with a modified argument

```
def factorial(n):  
    # Base case  
    if n == 0 or n == 1:  
        return 1  
    # Recursive case  
    else:  
        return n * factorial(n - 1)  
  
print(factorial(5))
```

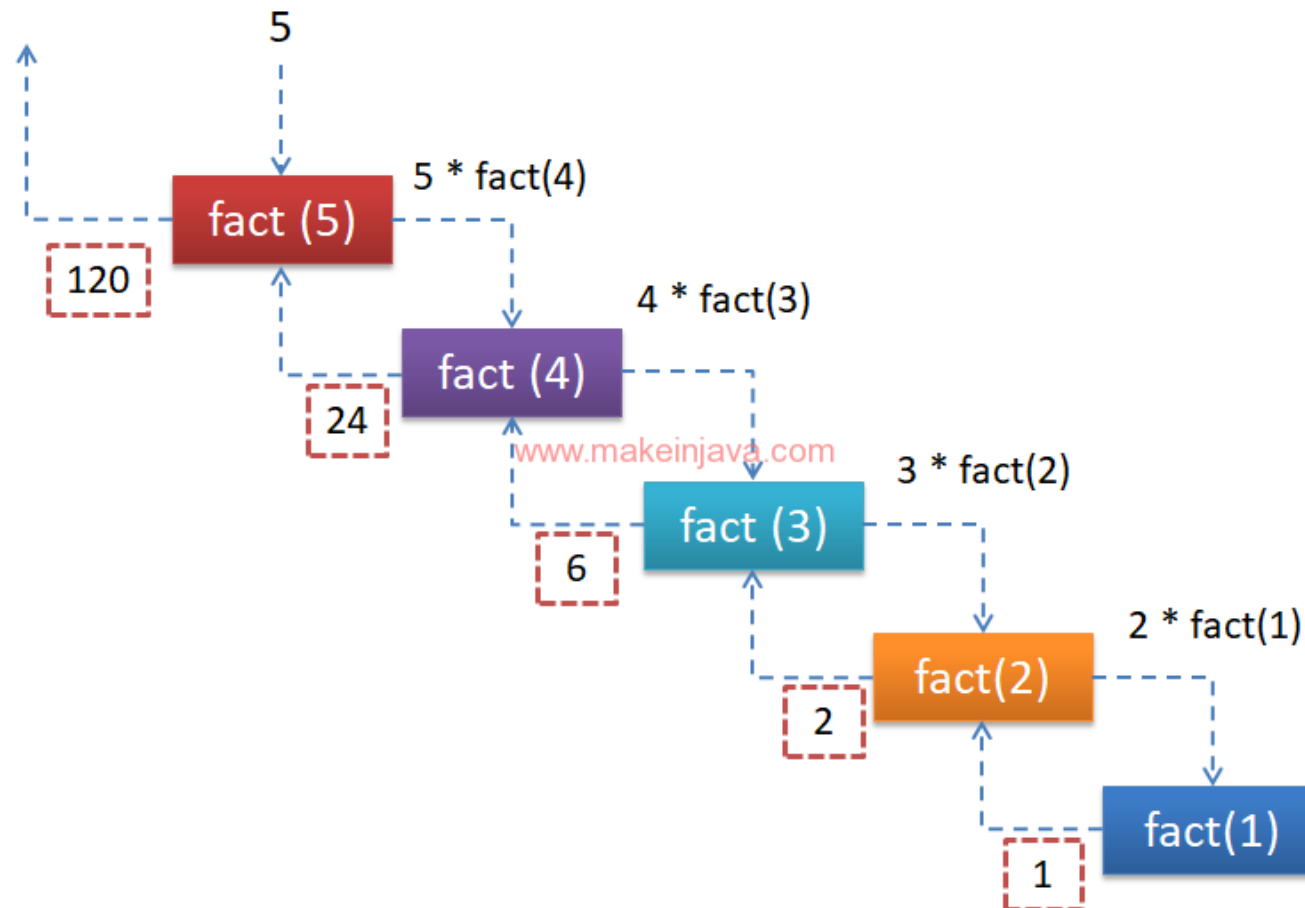


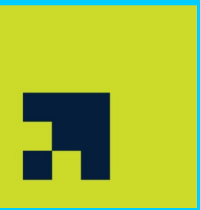
Recursion

[Hunt 2019]

[https://makeinjava.com/
find-factorial-number-java-
recursive-iterative-example/](https://makeinjava.com/find-factorial-number-java-recursive-iterative-example/)

26





Scope and Lifetime of Variables

- A variable is only available from inside the region it is created. This is called scope
 - Parameters and variables defined inside a function are not visible from outside
 - Hence, they have a local scope
 - The lifetime of a variable is the period throughout which the variable exists in the memory of our Python program
 - The lifetime of variables inside a function is as long as the function executes
 - These local variables are destroyed as soon as the function returns or terminates

```
def my_function():  
    a_local_variable = 100  
    print(a_local_variable)
```

```
# This line generates an error  
print(a_local_variable)
```



Global and Local Variables

- A variable created in the main body of the Python code is a global variable and belongs to the global scope
- Global variables are available from within any scope, global and local

```
x = 100
```

```
def my_function():  
    print(x)
```

```
my_function()
```




Naming Variables

```
x = 100
```

```
def my_function():  
    print(x)
```

```
my_function()
```

```
x = 100
```

```
def my_function():  
    x = 0  
    print(x)
```

```
my_function()  
print(x)
```



Global Keyword

- If we need to create a global variable, but are stuck in the local scope, we can use the global keyword.
- The global keyword makes the variable global.

```
def myfunc():  
    global x  
    x = 300
```

```
myfunc()
```

```
print(x)
```



Global Keyword

[Hunt 2019]

31

```
x = 300
```

```
def myfunc():  
    global x  
    x = 200
```

```
myfunc()
```

```
print(x)
```

Output?



The Nonlocal Keyword

[Hunt 2019]

32

- The *nonlocal* keyword is used to work with variables inside nested functions
- The *nonlocal* keyword makes the variable belong to the outer function

```
def myfunc1():  
    x = "Jane"  
    def myfunc2():  
        nonlocal x  
        x = "hello"  
    myfunc2()  
    return x  
  
print(myfunc1())
```